Session 05

Web Programming

Hands on frontend

Sessions Planning

Session 05

- Explain the required modules for the **first part** of the project.
- Start **implementing the Part 1** of the project.
- Publish the project to the **Github repository**.

Session 06

- Explain the required modules for the **second part** of the project.
- Start **implementing the Part 2** of the project.
- Publish the project to the **Github repository**.

Session 05

Angular Features

Features that will be used in the **first part** of the project.

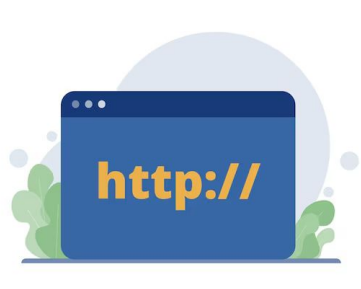
Angular HttpClient

Angular's `HttpClient` class is a powerful tool for making HTTP requests from an Angular application to a server over the network.

The `HttpClient` class simplifies the process of making HTTP requests by providing methods for commonly used HTTP methods like **GET**, **POST**, **PUT**, **DELETE**, etc.
It returns an **Observable** of the HTTP response, which allows for handling **asynchronous operations**.

It automatically performs JSON parsing for JSON responses.

Making HTTP GET Requests:



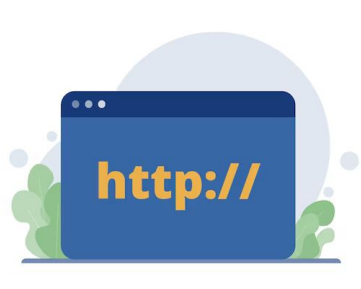

```
@Injectable()
export class CategoryService {

 private categoriesUrl = `http://localhost:8080/v1/categories`;

 constructor(private http: HttpClient) { }

 getAll(): Observable<Category[]> {
   return this.http.get<Category[]>(this.categoriesUrl);
 }
}
```

Making HTTP POST Requests:



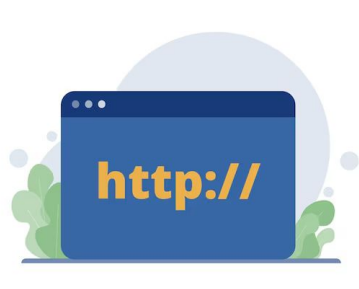

```
@Injectable()
export class CategoryService {

 private categoriesUrl = `http://localhost:8080/v1/categories`;

 constructor(private http: HttpClient) { }

 create(category: CategoryCreateInput): Observable<Category> {
   return this.http.post<Category>(this.categoriesUrl, category);
 }
}
```

Making HTTP PUT Requests:



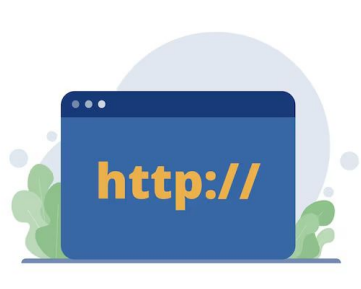

```
@Injectable()
export class CategoryService {

 private categoriesUrl = `http://localhost:8080/v1/categories`;

 constructor(private http: HttpClient) { }

 update(category: Category): Observable<Category> {
   return this.http.put<Category>(this.categoriesUrl, category);
 }
}
```

Making HTTP DELETE Requests:



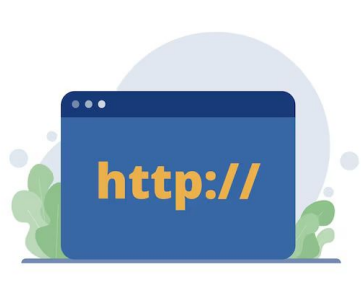

```
@Injectable()
export class CategoryService {

 private categoriesUrl = `http://localhost:8080/v1/categories`;

 constructor(private http: HttpClient) { }

 delete(category: Category): Observable<boolean> {
   return this.http.delete<boolean>(`${this.categoriesUrl}/${category.id}`);
 }
}
```

Handling Responses:




```
export class CategoryListComponent implements OnInit {

 categories: Category[] = [];

 constructor(private categoryService: CategoryService) { }

 ngOnInit(): void {
   this.categoryService.getAll().subscribe((categories) => {
     this.categories = categories;
   });
 }
}
```

Handling Errors:

```
@Injectable()
export class CategoryService {

 private categoriesUrl = `http://localhost:8080/v1/categories`;

 constructor(private http: HttpClient) { }

 update(category: Category): Observable<Category> {
   return this.http.put<Category>(this.categoriesUrl, category)
     .pipe(
       catchError(this.handleError<Category>('update', category))
     );
 }

 protected handleError<T>(operation = 'operation', result?: T) {
   return (error: any): Observable<T> => {
     console.error(`${operation} failed: ${error.message}`, error); // log to console
     // Let the app keep running by returning an empty result.
     return of(result as T);
   };
 }
}
```

Cross-Origin Resource Sharing (CORS)

CORS is a **security feature** that controls access to resources on a web server from a different domain.
It allows servers to specify **which origins are allowed to access their resources** through HTTP headers. When a web page makes a cross-origin request, the browser checks with the server to see if the request is permitted. If allowed, the server responds with the appropriate CORS headers, allowing the browser to access the requested resources.

CORS Configuration

Put simply, our **frontend**, which runs on **http://localhost:4200/**, aims to fetch data from a server running on **http://localhost:8080/**.

However, since these addresses don't match, the server will block the frontend's request.

To allow access from the frontend URL, we need to configure our backend to permit this new address to access its resources.

Java Spring Boot CORS Configuration

```java
package com.dauphine.blogger.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
@EnableWebMvc
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**").allowedOrigins("http://localhost:4200");
    }
}
```

Configuring Angular Environments

When developing applications, different environments are setup in order to test the application before the production deployment.
In angular, you can define different named build configurations for your project, such as **development** and **staging**, with **different defaults**.

The project's **src/environments/** directory contains the base configuration file, **environment.ts**, which provides configuration for **production**, the default environment. You can override default values for additional environments, such as development and staging, in target-specific configuration files.

Configuring Angular Environments

environment.ts (Default for dev)

EXPLORER

OPEN EDITORS

SAMPLE-APP

- src
  - app
    - components
    - data
    - environment
      - environment.prod.ts
      - environment.ts

TS environment.ts

src > app > environment > TS environment.ts > ...

You, 45 minutes ago | 1 author (You)

```
1 export const environment = {
2   production: false,
3   apiUrl: 'http://localhost:8080/'
4 };
5
```

environment.prod.ts (Default for dev)

EXPLORER

OPEN EDITORS

SAMPLE-APP

- src
  - app
    - components
    - data
    - environment
      - environment.prod.ts
      - environment.ts

TS environment.prod.ts

src > app > environment > TS environment.prod.ts > ...

You, 46 minutes ago | 1 author (You)

```
1 export const environment = {
2   production: true,
3   apiUrl: 'http://localhost:8080/'
4 };
5
```

Environment Variable Usage

In our application, we can utilize environment variables in the **HTTP Services**. Instead of manually typing **"http://localhost:8080/"** for the backend in every service, we can add it to the environment variable and use it throughout. It's essential because this URL will likely change when moving from a testing environment to a production one.

```
import { environment } from "../environment/environment";

You, 11 minutes ago | 1 author (You)
@Injectable()
export class CategoryService {

  private categoriesUrl = `${environment.apiUrl}v1/categories`;
```

Make sure to use the **“backtick”** or **“backquote”** **(Altgr-7)** and not the **“single quote”**.

BloggerBox Frontend Part 1

BloggerBox Frontend - Part 1

The objective of this project is to create a **user-friendly** Blogging Platform with key functionalities including **listing all blog posts** and enabling users to **post new ones** through an intuitive UI.

🚨 **Submission deadline:**

- **Classic: Mars 5th, by 11:59 PM.**
- **Alternance: April 17th, by 11:59 PM.**

It should be done individually and submitted via GitHub on a **separate repository** than the backend. Please add my GitHub user, **"bilal-elchami"**, as a contributor to your project in order to access the code.

How to invite collaborators to a GitHub project: GitHub Documentation

This project will count **20%** of your **final grade**.

BloggerBox Frontend - Part 1 Expected UI

Blogger-Box

Post

My first amazing title

My first amazing content post

Adoption May 7, 2024, 9:27:17 PM

The Transformative Power of Foster Care Adoption: Changing Lives, Building Families

Foster care adoption provides permanent homes for children who have experienced trauma or neglect, offering them a chance at stability and love. This article highlights the stories of families who have embraced foster care adoption and the profound impact it has had on both the children and their new families.

Adoption May 7, 2024, 9:27:17 PM

Open Adoption: Nurturing Relationships Beyond Blood Ties

Open adoption fosters transparency and ongoing contact between birth parents, adoptive parents, and the adopted child. This article explores the benefits of open adoption, including maintaining connections with birth families, navigating challenges, and supporting healthy relationships within the adoptive family unit.

Adoption May 7, 2024, 9:27:17 PM

Adopting Older Children: Overcoming Challenges, Embracing Opportunities

Adopting older children presents unique challenges and rewards. This article delves into the experiences of families who have

BloggerBox Frontend - Part 1

Part 1: Frontend Development

- **Create a Github repository for your Blogger Box Frontend App.**
- **Creation of Blogger Box Frontend Angular App:**
  - Utilize the provided second session slides to initiate the development of the Blogger Box frontend Angular application.
- **Implementation of the `PostService.ts`:**
  - Develop the Post Service responsible for fetching posts from the backend server.
- **CORS Configuration:**
  - Configure Cross-Origin Resource Sharing (CORS) on the backend to facilitate frontend access.

Sync with Github

BloggerBox Frontend - Part 1

- **Environment Variable Externalization:**
  - Externalize the API URL to an environment variable for enhanced flexibility and security.
- **Homepage Implementation:**
  - Design the homepage to retrieve and display all posts fetched from the backend server.
  - Use bootstrap elements to make the home page visually appealing.
  - Use the Angular date pipe to format the posts creation date into a user friendly date.

Sync with Github

Temporary backend API

For those who were unable to complete the required backend endpoints, you can utilize the following temporary API URL **(available only during class sessions)**:

API URL:

https://xs7c7x9j-8080.uks1.devtunnels.ms/

API Swagger Documentation:

https://xs7c7x9j-8080.uks1.devtunnels.ms/swagger-ui/index.html